INFORME METODOLÓGICO

Análisis de datos para Parking Tracker

Proyecto de Aula VI 2026-2 | Asignatura: Minería de Datos I

Fecha: 13 de octubre de 2026

Resumen

Este informe documenta la revisión inicial del código de Parking Tracker y plantea una metodología para analizar los registros de entrada de vehículos. El sistema ya registra entradas y salidas, actualiza el estado de los puestos y genera facturas. Además, incluye una consulta agregada de entradas por hora durante los últimos siete días. En esta versión se documenta la arquitectura y se propone el flujo de minería de datos; el análisis exploratorio sobre un archivo de datos y el entrenamiento del modelo quedan pendientes de ejecución y evidencia.

1. Introducción y problema

Parking Tracker es una aplicación para consultar parqueaderos y puestos, registrar entradas y salidas y administrar usuarios. El problema de análisis seleccionado es la falta de una estimación futura de la demanda horaria. La pregunta de trabajo es: ¿pueden los registros históricos de entrada ayudar a estimar cuántos vehículos ingresarán en cada hora de un parqueadero?

Objetivo del análisis: evaluar la viabilidad de predecir el conteo de entradas por hora para apoyar la identificación de horas pico y la planificación operativa.

2. Descripción del sistema y fuente de datos

La revisión del código identifica un backend con Node.js y Express, persistencia en MongoDB mediante Mongoose y una interfaz web basada en React y Vite. El frontend incluye Axios para peticiones HTTP, React Router para navegación, Firebase y Socket.IO Client. El backend usa Socket.IO para emitir cambios de estado de los puestos.

Los modelos relevantes son RegistroEntrada (puesto, usuario, placa, horaEntrada, horaSalida y estado), Puesto (número, estado, tipo, zona, nivel y parqueaderoId), Parqueadero (nombre, dirección, tarifa por minuto y estado) y Factura (tiempo total en minutos, valor cobrado y fecha de emisión). Para el análisis temporal, la variable central es horaEntrada y la asociación con Puesto permite identificar el parqueadero.

El código contiene scripts de demostración que crean registros históricos sintéticos de los últimos siete días y ocupaciones activas aleatorias. Estos datos son útiles para pruebas técnicas, pero no deben confundirse con observaciones operativas reales. No se ha confirmado en esta revisión qué conjunto de datos está actualmente conectado a la aplicación.

3. Metodología y uso de MinedApp

Se adopta la secuencia de seis etapas indicada en la guía del curso: problema, datos, preprocesamiento, modelado, evaluación y reporte. La revisión de código permite formular el problema y localizar la fuente potencial de datos. Para completar las etapas restantes se debe exportar una muestra autorizada a CSV, cargarla en MinedApp y conservar evidencias de las decisiones y resultados.

3.1 Problema

Definir la tarea como regresión del número de entradas por hora. El nivel de agregación será parqueadero-hora.

3.2 Datos

Exportar los registros necesarios, revisar las primeras filas, dimensiones, tipos de variables, valores nulos, duplicados y cobertura temporal. Documentar si el dataset es real, sintético o mixto.

3.3 Preprocesamiento

Convertir las fechas a un formato consistente y zona horaria definida; comprobar registros sin fecha o con fechas imposibles; agregar por parqueadero y hora; crear variables hora del día y día de semana. Tratar anomalías solo tras investigar su causa.

3.4 Modelado

Establecer una línea base sencilla y comparar con un modelo inicial de regresión, por ejemplo RandomForestRegressor. Mantener una división cronológica de entrenamiento y prueba.

3.5 Evaluación

Calcular MAE y RMSE sobre periodos posteriores no utilizados en el entrenamiento; comparar con la línea base e interpretar los errores en número de entradas.

3.6 Reporte

Guardar el notebook, el código de preparación, las métricas reales, las visualizaciones pertinentes y las limitaciones en el repositorio.

4. Exploración y preparación de datos

En la revisión del código se verificó que el endpoint ocupacionPorHora filtra registros de los últimos siete días, relaciona cada registro con su puesto y agrupa por hora usando la zona horaria America/Bogota. Devuelve 24 categorías horarias y el número de entradas en cada una. Esta funcionalidad sirve para exploración descriptiva, pero no se dispone aquí de una exportación de la base de datos ni de estadísticas calculadas sobre un dataset ejecutado.

Por tanto, quedan pendientes de medir: número de registros y columnas, periodo cubierto, porcentaje de valores faltantes, duplicados, distribución de entradas por hora y diferencias entre parqueaderos. No se incluyen cifras inventadas ni conclusiones estadísticas no comprobadas.

5. Modelo y evaluación

La propuesta es predecir cantidad_entradas_hora, una variable numérica discreta creada mediante el conteo de entradas en cada intervalo. Las variables candidatas son hora del día, día de la semana, parqueadero y conteos de horas previas, siempre que se construyan sin utilizar información futura. La capacidad total de puestos podría añadirse como variable contextual.

Se recomienda comparar una línea base con un modelo inicial como RandomForestRegressor. Los datos deben separarse cronológicamente. Las métricas propuestas son MAE y RMSE. El modelo no se considera construido ni validado hasta que el código se ejecute y se registren las métricas reales sobre datos de prueba.

6. Resultados e interpretación

Resultado de la revisión de software: existe un endpoint descriptivo de entradas por hora para los últimos siete días, junto con modelos de datos para entradas, puestos, parqueaderos y facturas. También existe un script que genera historial sintético para demostración. Estos hallazgos describen funcionalidades presentes en el código; no representan resultados de un experimento predictivo.

Resultados de minería de datos: pendientes. Se deben incorporar tablas y gráficas obtenidas al ejecutar el análisis, junto con el número de registros, patrones observados y métricas de evaluación. No debe afirmarse un porcentaje de precisión ni una mejora del modelo sin evidencia reproducible.

7. Limitaciones y conclusiones

La aplicación dispone de una base técnica útil para iniciar el análisis, pero la disponibilidad de modelos de datos no garantiza por sí sola que exista suficiente información histórica para entrenar un modelo fiable. El historial de demostración es sintético y cubre siete días, por lo que no debe usarse como prueba de demanda real ni de generalización. Se requiere verificar la fuente, volumen, calidad y autorización de uso de los registros.

Como siguiente paso se recomienda exportar datos reales autorizados (o etiquetar explícitamente un dataset sintético para pruebas), ejecutar la exploración en MinedApp o Python, construir la variable objetivo por hora, entrenar y comparar modelos, y actualizar este informe con resultados comprobables. La predicción sería un apoyo a decisiones operativas, no una garantía de disponibilidad de espacios.

8. Referencias y repositorio

- Proyecto Parking Tracker, código fuente del frontend y backend entregado para la revisión.
- Guía práctica para completar los entregables del Proyecto de Aula, Minería de Datos I, proporcionada por el curso.
- Formato “Propuesta modelo predictivo proyecto de aula”, proporcionado por el curso.

Repositorio: [Agregar enlace al repositorio del proyecto]

Lista de evidencias pendientes para la versión final

- Enlace funcional al repositorio y nombres/números de clase de los integrantes.
- Archivo CSV autorizado y descripción de su procedencia.
- Capturas o exportación del análisis exploratorio en MinedApp.
- Código ejecutado de preprocesamiento y modelado.
- Métricas reales (MAE/RMSE) y comparación con una línea base.
- Gráficas y conclusiones derivadas de los resultados observados.